

Multi-view based network for malware classification

Lijie Chu^{a,b,1}, Yiqian Zhang^{a,b,1}, Yuexi Qiu^{a,b,1}, Jing Xing^b, Changjian Zhou^{b,*}

^a School of Electrical and Information, Northeast Agricultural University, Harbin, 150030, PR China

^b Department of Data and Computing, Northeast Agricultural University, Harbin, 150030, PR China

ARTICLE INFO

Keywords:

Malware classification
Cyberspace security
Vision transformer
Self-attention
Deep learning

ABSTRACT

Malware poses grave threats to cyberspace security, necessitating prompt and precise classification. However, existing static analysis methods often overlook the alignment sensitivity inherent in mapping one-dimensional (1D) binary sequences to three-dimensional (3D) chromatic image data, leading to feature loss due to phase shifts. Furthermore, the effective fusion of local byte-level textures and global assembly automatic storage management (ASM) semantics remains a challenge. To address these limitations, this study proposes an innovative Multi-view based Network that redefines “multi-view” feature extraction at two complementary levels. Microscopically, we introduce a bit-shifting strategy to generate eight shifted views from each binary sequence, exhaustively capturing all local red, green, and blue (RGB) alignments to ensure feature invariance against obfuscation. Macroscopically, we integrate these raw byte-level views with the global ASM view, fusing fine-grained binary textures with high-level instruction semantics. The framework leverages a Vision Transformer (ViT) and Visual Geometry Group (VGG) via Transfer Learning, and employing a Bilinear Attention Network (BAN) to model high-order interactions between these modalities. Experimental results demonstrate the superiority of the proposed approach, which achieves a significant performance improvement over state-of-the-art baselines and thus validates the effectiveness of the dual-level multi-view strategy for decoding complex malware binary files.

1. Introduction

Malware intended to disrupt, damage, or unlawfully control computer systems usually consists of malicious code hidden within seemingly harmless programs, posing a severe threat to individual users, businesses, and national security (O’Kane et al., 2011). Accordingly, malware has been expanding swiftly with the rapid growth of the software industry and the endless emergence of security vulnerabilities in networks. Consequently, developing an efficient approach to detect and classify massive malware has become particularly important.

In recent years, malware detection approaches are mainly divided into two streams: dynamic analysis and static analysis. Dynamic analysis identifies malicious behavior by observing the operational behavior of software, like sandbox testing. Inoue et al. have implemented a fully automatic malware analysis system through a sandbox, enabling large-scale malware analysis (Inoue et al., 2009). Chen and Bridges conducted analyses of ransomware, including WannaCry and its polymorphic variants, through log file generation within a sandbox environment

(Cuckoo Sandbox) (Chen and Bridges, 2017). However, this approach presents challenges such as significant resource consumption and the complexity of replicating condition-specific behaviors. Conversely, static analysis entails extracting details like file metadata, strings, and function calls from samples without their execution, enabling preemptive identification of behavioral patterns and characteristics. Researchers can classify malware based on its static features using deep learning algorithms. For example, Hassen et al. proposed a new feature based on control statement tiling, it has comparable accuracy to features based on conventional opcode n-grams but requires a smaller size, meaning less training time is needed (Hassen et al., 2017). However, static analysis requires specialized domain knowledge and extensive feature engineering, making the process cumbersome.

Subsequently, to reduce feature engineering and improve the accuracy of detection against different malware variants, deep learning techniques are widely used for malware classification tasks. A significant approach is converting malware *byte* files into grayscale images and classifying them by deep neural networks, demonstrating the potential

* Corresponding author.

E-mail address: zhouchangjian@neau.edu.cn (C. Zhou).

¹ These authors contributed equally to this work.

of image processing techniques in malware research (Kalash et al., 2018; Nataraj et al., 2011). However, the grayscale image representations have limitations. Important characteristics that might differentiate one piece of malware from another could be lost in this translation, reducing the effectiveness of pattern recognition and feature extraction processes crucial for machine learning models, particularly deep learning models like convolutional neural networks (CNNs). Some researchers adopt color images to enrich feature information, as color images represent malware variants and other features better (Vasan et al., 2020). However, the accuracy of using deep learning models to classify color images is still not satisfactory. Consequently, some researchers rotate and cut color images to get more data so that malware detectors can better generalize unseen data (Singh et al., 2019). But these data enhancement methods can introduce features that are not relevant to the malware itself or hide important features, thus affecting the classification results. To address this, recent studies have shifted toward ViT, which excel at capturing global dependencies and long-range correlations in malware binaries that are often missed by the local receptive fields of standard CNNs (Ashawa et al., 2024). Furthermore, new transformer-based models like MalFormer have demonstrated that attention mechanisms can significantly enhance robustness against local obfuscations compared to traditional architectures (Jeong et al., 2025).

Overall, existing approaches have always suffered from the following limitations. First, the RGB Alignment Sensitivity problem is largely overlooked. Most methods map 1D binary streams directly to 3D images ignoring that a pixel consists of three bytes representing the red, green, and blue channels. Consequently, a trivial shift such as a single byte insertion can disrupt the byte-to-pixel alignment, drastically altering visual textures and causing feature loss. Second, current approaches suffer from ‘Modal Isolation’. They typically rely solely on either the raw byte stream, which captures texture but lacks logic, or disassembled ASM code, which captures semantics but loses obfuscation details. This isolation results in a failure to effectively model the high-order complementary interactions between these two modalities. To tackle these challenges, this paper proposes a Multi-view based Network utilizing ViT and VGG via Transfer Learning and bilinear attention fusion. The main contributions are summarized as follows.

- (1) Dual-level Multi-view Framework. We redefine “multi-view” at two complementary levels. Microscopically, we propose an 8-view bit-shifting strategy to resolve RGB alignment sensitivity, which ensures feature invariance against phase shifts. Macroscopically, we integrate the raw Byte-stream view with the ASM-stream view to achieve semantic and textural completeness.
- (2) Bilinear Attention Fusion. We employ a Bilinear Attention Network (BAN) to model the complex high-order interactions between visual texture features extracted by VGG and semantic assembly features extracted by ViT. This approach significantly outperforms simple concatenation methods.
- (3) State-of-the-Art Performance. We conducted extensive experiments on the Microsoft Big2015 dataset. The proposed method achieves an accuracy of 98.78% and an F1-score of 98.76%, surpassing state-of-the-art baselines and validating the effectiveness of the proposed approach.

2. Related work

2.1. Gray and color images for malware classification

Converting malware binary files to grayscale images is proven to be effective in malware classification tasks. This method maintains the structural information of the original binaries and emphasizes the visual distinctions between different malware types, setting the stage for subsequent feature extraction and classification. Nataraj et al. initially explored this technique, transforming malware binaries into grayscale images to make structural and pattern differences between samples

visible (Nataraj et al., 2011). Dai et al. further endorsed this technique by transforming malware memory dumps into grayscale images and classifying them by multilayer perceptron (MLP), proving its effectiveness (Dai et al., 2018). Additionally, Bakour and Ünver developed a framework that merges machine learning classifiers with deep neural networks, attaining a 98.2% accuracy in detection, thus highlighting the benefits of grayscale image conversion for malware classification (Bakour and Ünver, 2021).

However, while converting images to grayscale maintains the original structure, it may not fully capture the global features of the malware, particularly for complex malware that exhibits subtle behavioral patterns not easily discernible in grayscale. This limitation can result in the loss of crucial information, potentially impacting classification accuracy. In contrast, color images can potentially provide richer information for representing malware features and variants. Several studies have demonstrated this advantage: Ni et al. (2018) showed that RGB malware visualization preserves more structural information compared to grayscale representations, achieving 4.2% higher classification accuracy. Similarly, Yakura et al. (2018) demonstrated that color-based visualization enables more effective detection of malware variants through preserving byte-range specific features. Gibert et al. (2020) further showed that RGB representations help to distinguish subtle differences between malware families that might appear similar in grayscale. For instance, Naeem et al. developed a method to convert malware into color images and employed deep convolutional neural networks for identification (Naeem et al., 2020). Vasan et al. synthesized color images and utilized deep CNNs for classification, also achieving notable accuracy (Vasan et al., 2020). Compared to grayscale images, color images offer richer information and more accurately characterize malware features and variants, thereby enhancing classification accuracy.

2.2. Opcodes for malware detection

In malware detection tasks, opcode analysis is a critical static analysis technique for identifying malicious behaviors. Jeon and Moon introduced a novel malware detection model using a convolutional recurrent neural network on opcode sequences, achieving 96% accuracy (Jeon and Moon, 2020). Moskovitch et al. proposed a method for detecting unknown malicious codes by analyzing opcodes in executable files and using n-grams as features in the classification process, achieving an accuracy of over 99% (Moskovitch et al., 2008). Saleh reviewed malware detection methods based solely on opcode features and observed a positive correlation between detection rates and the year of study (Saleh, 2023). Zhang et al. introduced a new static analysis-based method for classifying ransomware, achieving 91.43% accuracy rate (Zhang et al., 2019). However, these studies often focus on the syntactic analysis of opcodes, which, while informative, may not encompass the entire spectrum of malware behavior.

2.3. Breaking single-view constraints

Recent deep learning models have improved malware classification. For example, FACILE (Zou et al., 2024) uses capsule networks to capture spatial hierarchies, SDIF-CNN (Kumar and Panda, 2023) combines features from multiple CNNs (e.g., VGG, ResNet, DenseNet), and MobileNet-based methods (Palma Salas et al., 2023) offer lightweight inference.

However, most approaches rely on grayscale or single-channel images, which may miss key structural and behavioral features. While some studies use color images, few explore systematic multi-view generation from a single binary, risking loss of variant-specific traits. Moreover, combining byte-level and assembly-level (ASM) information is rare, limiting the semantic richness of representations. Feature fusion methods like concatenation or averaging often fail to capture complex cross-modal dependencies. Conversely, hybrid architectures emerging recently have demonstrated that combining the local textural feature

extraction of CNNs with the global context awareness of Transformers significantly improves classification accuracy on complex datasets (Alshomrani et al., 2025).

To address these gaps, we propose a unified multi-view framework. It generates multiple color views via fine-grained displacement, extracts features using transfer learning, and integrates byte and ASM modalities through bilinear attention. This design improves accuracy and better captures malware diversity compared to existing models.

3. Proposed method

3.1. Problem definition and notation

We consider the task of multi-class malware classification based on static analysis data. Let the dataset be defined as $\mathcal{D} = \{(B_i, A_i, y_i)\}_{i=1}^N$ where B_i denotes the byte-level file, A_i denotes the corresponding assembly-level (ASM) file of the i -th malware sample, and $y_i \in \{1, 2, \dots, C\}$ is its class label among $C = 9$ malware families. The goal is to learn a classification function f_θ parameterized by θ that maps each input pair to a predicted class:

$$f_\theta(B_i, A_i) \rightarrow \hat{y}_i. \quad (1)$$

To facilitate image-based learning, each byte file B_i is first transformed into a set of K RGB images, denoted as:

$$\mathbf{X}_i = \mathbf{x}_i^{(1)}, \mathbf{x}_i^{(2)}, \dots, \mathbf{x}_i^{(K)}, \mathbf{x}_i^{(k)} \in \mathbb{R}^{H \times W \times 3} \quad (2)$$

where each $\mathbf{x}_i^{(k)}$ is generated by applying a fine-grained offset (byte-shift) to the 8-bit byte sequence, and $K = 8$ in our framework. This multi-view design aims to preserve local structural variance and partially mitigate the loss of sequential information inherent in image transformation.

Simultaneously, the ASM file A_i is parsed to extract opcode-level statistics. Specifically, we compute the term frequency (TF) vector of d predefined opcodes:

$$\mathbf{o}_i = \text{TF}(A_i) \in \mathbb{R}^d, \quad (3)$$

and convert it into a single RGB image $\mathbf{z}_i \in \mathbb{R}^{H \times W \times 3}$ via color mapping.

We then apply deep pretrained feature extractors using transfer learning. The K byte images $\mathbf{X}_i$ are passed through a fixed VGG16 encoder $\phi_{\text{byte}} : \mathbb{R}^{H \times W \times 3} \rightarrow \mathbb{R}^{d_1}$, producing

$$\mathbf{f}_i^{\text{byte}} = \text{Concat}(\phi_{\text{byte}}(\mathbf{x}_i^{(1)}), \dots, \phi_{\text{byte}}(\mathbf{x}_i^{(K)})) \in \mathbb{R}^{K \cdot d_1}. \quad (4)$$

The ASM image $\mathbf{z}_i$ is processed by a pretrained Vision Transformer $\phi_{\text{asm}} : \mathbb{R}^{H \times W \times 3} \rightarrow \mathbb{R}^{d_2}$ to obtain

$$\mathbf{f}_i^{\text{asm}} = \phi_{\text{asm}}(\mathbf{z}_i) \in \mathbb{R}^{d_2} \quad (5)$$

To integrate these two modalities, we adopt a Bilinear Attention Network (BAN), which models high-order interactions between byte-level and ASM-level features. The fused representation is defined as

$$\mathbf{f}_i = \text{BAN}(\mathbf{f}_i^{\text{byte}}, \mathbf{f}_i^{\text{asm}}) \in \mathbb{R}^d \quad (6)$$

where BAN computes attention-guided bilinear pooling between the two feature sets, capturing cross-modal dependencies.

Finally, a multi-layer perceptron (MLP) classifier is applied to obtain the class probabilities

$$\hat{\mathbf{y}}_i = \text{MLP}(\mathbf{f}_i) \in \Delta^C, \quad (7)$$

where Δ^C is the C -dimensional probability simplex. The model is trained to minimize the categorical cross-entropy loss

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^C \mathbf{1}(y_i = c) \cdot \log \hat{y}_i^{(c)}. \quad (8)$$

The mathematical symbols and notations used in this paper are summarized in Table 1.

3.2. Multi-view image generation

As illustrated in Fig. 1, the proposed approach generated corresponding images for malware binary sequences using a step size of 1. Specifically, we converted the *byte* files into binary strings and further partitioned them into numerous 8-bit substring patches. Subsequently, these substrings were transformed into unsigned integers ranging from 0 to 255, forming one-dimensional feature vectors. To extract visual information, we mapped these feature vectors to RGB pixels. To capture richer data patterns and compensate for potential data corruption, we employed a displacement strategy with a step size of 1, generating eight slightly varied versions for each binary string. This approach ensures more comprehensive and nuanced representation of the visual features, providing richer and more accurate information for subsequent analysis. To address the varying sizes of malware binaries, we implemented a consistent normalization procedure. We explicitly chose zero-padding over image resizing or dimensional reduction to preserve the integrity of the original malware binaries. Since deep learning models utilizing Transfer Learning (e.g., VGG16) require a fixed input resolution, resizing smaller binaries to fit this dimension would require pixel interpolation. In the context of malware visualization, where each pixel strictly corresponds to a specific byte value (instruction), interpolation would alter the pixel values, effectively corrupting the original machine code instructions. In contrast, zero-padding appends non-informative values to the end of the sequence without altering the original byte stream. Coupled with the self-attention module in our architecture, the model is designed to effectively focus on valid code sequences while assigning negligible attention weights to the padded regions.

3.3. Model architecture

The overall framework of the proposed approach is illustrated in Fig. 2, while Algorithm 1 provides a pseudocode-based workflow to further clarify the step-by-step simulation process. Two primary deep learning models are integrated for classification tasks: the vision transformer and VGG16. Transfer learning is leveraged to utilize them as feature extractors due to the high cost of fine-tuning. Specifically, we employ the first 17 layers of the pre-trained VGG16 model to harness its capability in processing early to mid-level image features. To grasp the key features that contribute most to the positive predictions, the proposed method introduces an attention module that utilizes a self-attention to reveal local connections in color images. Additionally, the proposed method incorporates dropout layers to mitigate overfitting during the learning process. The subsequent sections of the VGG16 model, starting from the third max-pooling layer, are also utilized in a

Table 1
Mathematical symbols.

Symbol	Description
B_i	Byte file of the i -th malware sample
A_i	ASM (assembly) file of the i -th malware sample
y_i	Ground truth class label of sample i , from $\{1, \dots, C\}$
$\hat{y}_i$	Predicted class probability vector for sample i
C	Total number of malware classes (families)
f_θ	Classification function (deep model) with learnable parameters θ
$\mathbf{X}_i^B$	Set of RGB images generated from byte file B_i
$\mathbf{X}_i^A$	Color image generated from ASM file A_i
L	Loss function used for training (categorical cross-entropy)
PO	Number of occurrences of a particular opcode
AO	Total number of occurrences of all opcodes
p_i^j	Probability that sample i belongs to class j
$\mathcal{D}$	Training dataset $\{(X_i, y_i)\}_{i=1}^N$
θ	Learnable parameters of the model

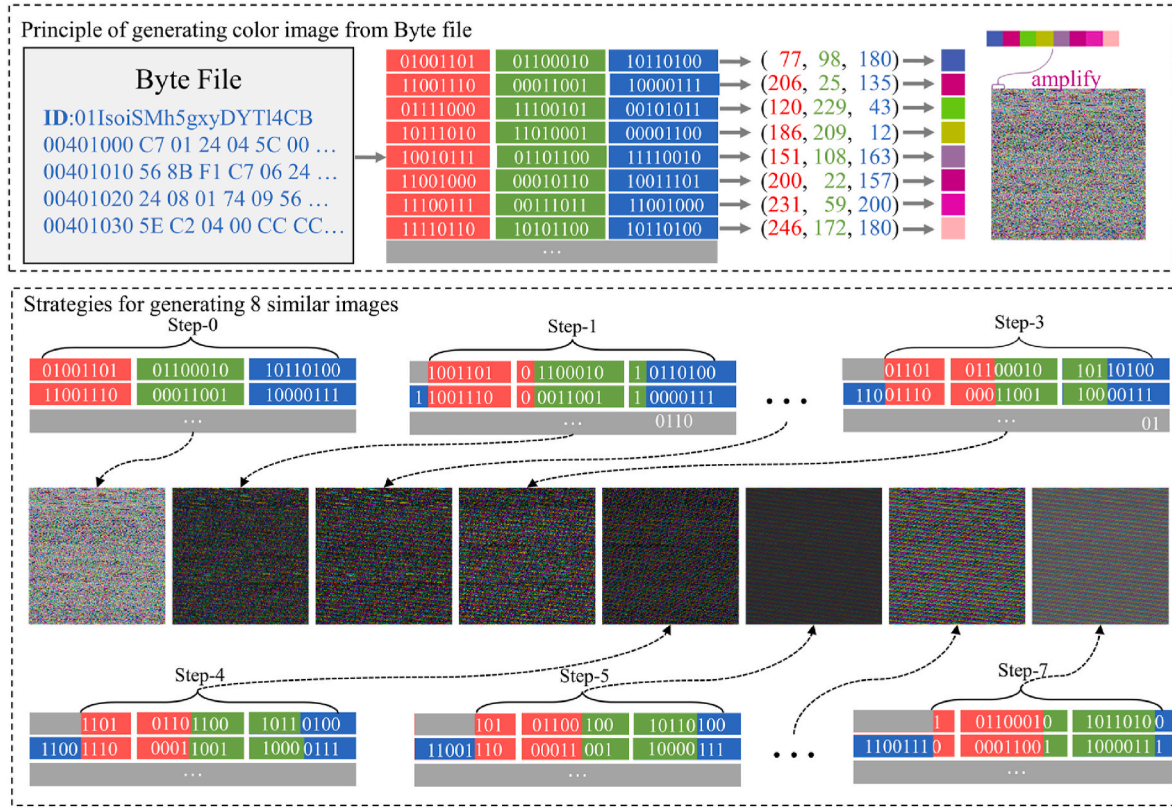


Fig. 1. The process of multi-view image generation.

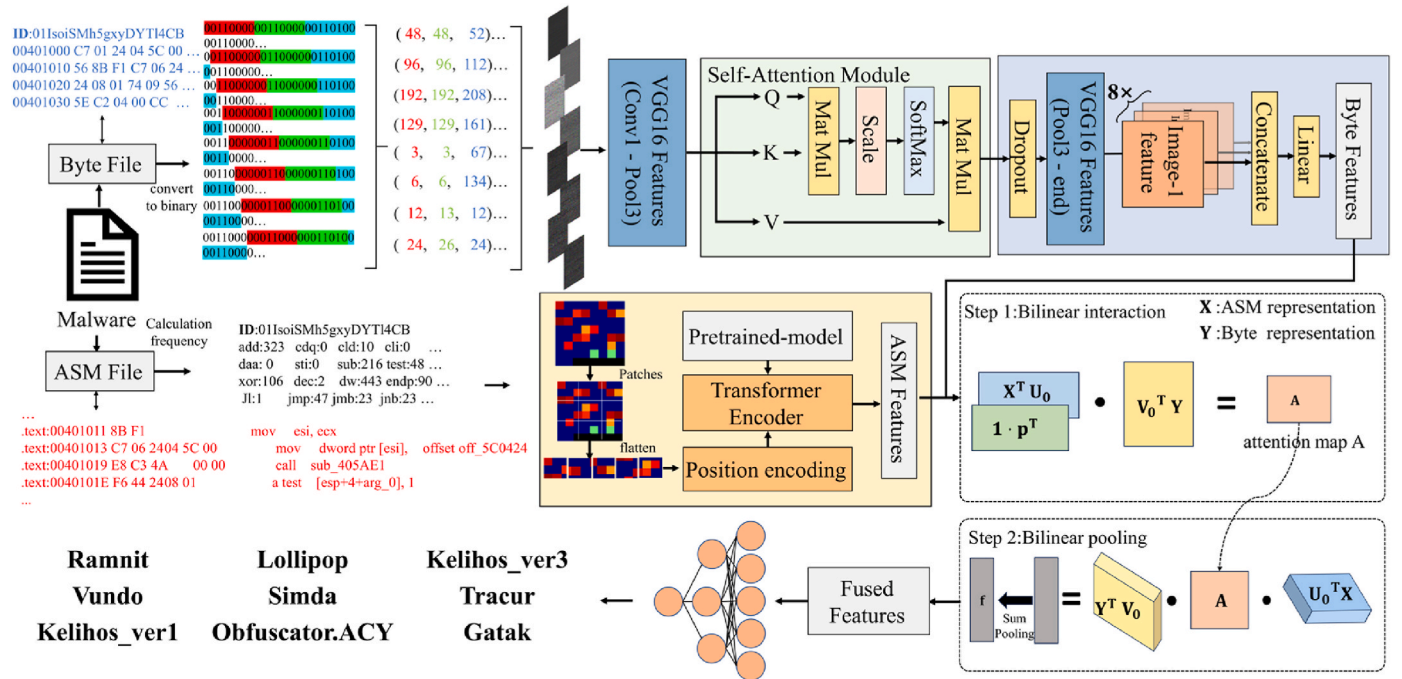


Fig. 2. The overall architecture of the proposed method.

pre-trained fashion to extract features related to malware binary files. Given that each type of malware is represented by 8 different images, the proposed method concatenates these images to ensure no crucial features are missed. To standardize the varying sizes of malware images, each image is resized to 224×224 in RGB format, aligning with the

common input image size of 224×224 used in deep learning architectures, as established in prior studies.

Algorithm 1. Multi-view Image Generation and Classification

Algorithm 1 Multi-view Image Generation and Classification

Require**1: Input**

- binary sequence: The binary sequence of the malware file.
- asm file: The ASM file of the malware.

Ensure:**2: Output:**

- predicted family: The classified malware family.

3: begin**4: I. Multi-view Image Generation****5: for** all binary sequence **do**

- 6: **Call** Generate Multi View Images procedure;
- 7: i) Shift the binary sequence with step size 1 to generate 8 varied images;
- 8: ii) Convert each 8-bit substring to decimal values;
- 9: iii) Map decimal values to RGB pixels to form images;
- 10: iv) Save images in 224 × 224 RGB format;

11: end for**12: II.FeatureExtraction****13: for** all images **do**

- 14: **Call** ExtractFeatures procedure;
- 15: i) Extract byte features using pre-trained VGG16
- 16: ii) Extract ASM features using pre-trained ViT;

17: end for**18: III. Feature Fusion with Bilinear Attention Network (BAN)**

- 19: **Call** FuseFeatures procedure;
- 20: i) Calculate bilinear interactions between byte and ASM features;
- 21: ii) Normalize attention weights using softmax function;
- 22: iii) Sum channel dimensions to obtain fused features;

23: IV. Classification

- 24: **Call** Classify procedure;
- 25: i) Classify fused features using MLP;
- 26: ii) Return predicted family;

27: end

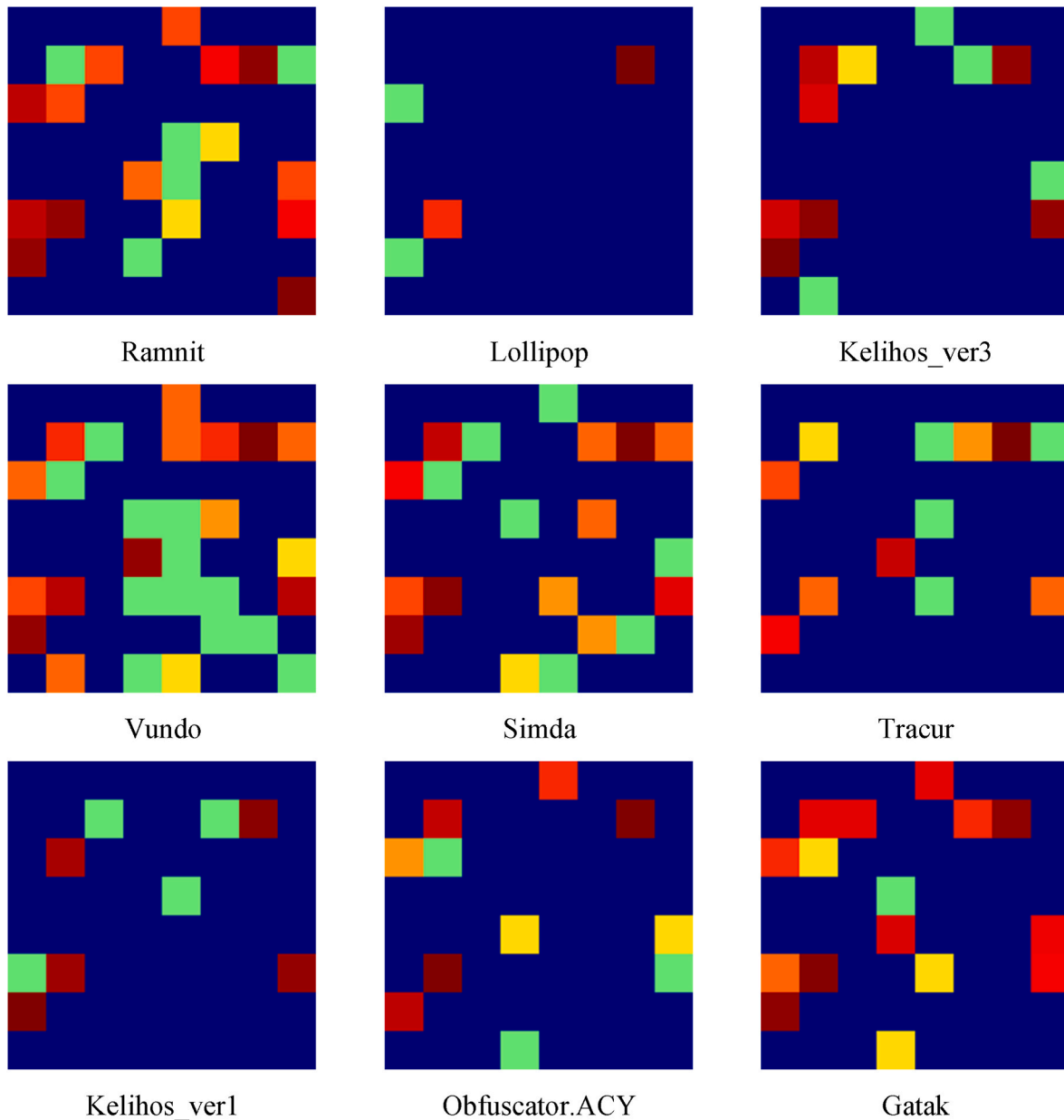


Fig. 3. ASM charts for different malware families.

Next, utilizing a pre-trained ViT model for feature extraction from images generated by ASM. Finally, the two sets of features were fused using BAN to condense both the front-end information of byte files and the global information of ASM files. The resulting output features were 1024-dimensional, which were then classified into malware families using MLP.

3.3.1. Feature extraction of ASM files

This study focuses on transforming opcode features from malware into color images, enabling the extraction of malware features through deep learning techniques. In selecting opcodes, we consulted the dataset available on Kaggle ([BIG Malware Dataset from Microsoft](#)), which listed 66 opcodes. To accommodate the ViT model, we excluded the two most frequently occurring opcodes (add and call), finalizing a set of 64 opcodes. These opcodes were then mapped to 224×224 pixels color images based on their frequency information, facilitating the training and testing of deep learning models.

opcodes = ["cdq", "cld", "cli", "cmc", "cmp", "cwd", "daa", "sti", "stos", "sub", "test", "wait", "xchg", "xor", "dd", "dec", "dw", "endp", "faddp", "fchs", "fdivr", "fdivr", "fistp", "fld", "fstp", "fword", "fxch", "imul", "in", "inc", "ins", "jb", "je", "jg", "jl", "jmp", "jnb", "jno", "jo", "jz", "lea", "mov", "mul", "not", "or", "out", "outs", "pop", "push", "rcl", "rcr", "rep", "ret", "rol", "ror", "sal", "sar", "sbb", "scas", "shl", "shr", "sidt", "stc", "std", "sti", "stos", "sub", "test", "wait", "xchg"]

In this section, each opcode generates a colored image block, creating an 8×8 grid that corresponds directly to the ViT model used in later training.

To obtain the malware opcode information, we utilized `os` and `re` in Python libraries to read `.asm` files. The aim is to transform this comprehensive data for use in visual tasks. Since certain opcodes appear more frequently in specific malware families and less in others, studies suggest that lower frequency opcodes may reflect unique family characteristics of malware. Therefore, it is essential to calculate the Term Frequency (TF) of each opcode in the dataset. TF is defined as follows

$$TF(t) = \frac{PO}{AO} \quad (9)$$

TF stands for the frequency of the occurrence in a document. PO stands for the number of occurrences of a particular opcode in that malware. AO stands for the number of occurrences of all opcodes in a particular malware. To adapt the term frequency values for image representation, we normalize each TF value by multiplying it by 255. Then, we further process these values using a transformation formula to enhance the visual appeal of the images.

$$NormalizedValue = TF * 255 \quad (10)$$

$$TransformedValue = \frac{255}{1 + NormalizedValue} \quad (11)$$

Here, the normalized value refers to the TF value multiplied by 255. These normalized values are then converted to RGB colors using a color mapping function. We use the plt.cm.jet color map with inversion, making higher opcode values appear as cooler colors in images to increase visual contrast. Since opcodes are uniquely mapped to software, the resulting color images intuitively show the relative frequency of different opcodes in the sample. Fig. 3 shows the ASM graph of a particular malware from different malware families.

3.3.2. Feature extraction of byte files

Algorithm 2. Multi-view RGB Image Generation from Malware Byte File

The pseudocode for our multi-view image generation process is illustrated in Algorithm 2, with detailed steps described below.

Read the byte files of malware and convert the file contents into binary string format. For instance, take a binary string b like '01100010 10011110 11010101 00010110 ...', the proposed method breaks it down into substrings of length 8

$$b = b_1 b_2 \dots b_n \quad (12)$$

Using the example, with $b_1 = 01100010$, $b_2 = 10011110$, and so on, each 8-bit substring b_i is transformed into an unsigned integer d_i , which falls within the range of 0 to 255, by applying the formula below

$$d_i = \sum_{j=1}^8 b_{ij} \times 2^{8-j} \quad (13)$$

Here, b_{ij} refers to the j th bit of b_i . Through this conversion, we obtain a one-dimensional feature vector from the binary file, where each element is the decimal value of an 8-bit substring.

Next, to construct images, we map these decimal values to RGB pixels. Specifically, every three consecutive decimal values (d_i, d_{i+1}, d_{i+2}) are mapped to one RGB pixel.

$$RGB_i = (d_i, d_{i+1}, d_{i+2}) \quad (14)$$

To capture richer patterns in the data and offset potential impacts from data corruption, we implemented a fine-tuning strategy. For each binary string, we create adjusted binary strings b_k based on an offset value k , where $0 < k < 8$.

Algorithm 2 Multi-view RGB Image Generation from Malware Byte File

Require: Byte file B_i

Ensure: Set of 8 RGB images $X_i^B = \{I_1, I_2, \dots, I_8\}$

```

1: Read byte file  $B_i$  and convert to binary string  $b = b_1 b_2 \dots b_n$ , where each  $b_i$  is 8 bits
2:   for  $k = 0$  to  $7$  do                                     ▷ Generate 8 shifted binary strings
3:     Create shifted binary string  $b^{(k)} = b_{k+1} b_{k+2} \dots b_n$ 
4:     Initialize empty list  $D = []$                                ▷ Decimal values list
5:     for  $i = 1$  to  $\text{length}(b^{(k)})$  do
6:       Extract 8-bit substring  $b_i^{(k)}$ 
7:       Convert  $b_i^{(k)}$  to decimal  $d_i = \sum_{j=1}^8 b_i^{(k)} \times 2^{8-j}$        ▷ Equation (5)
8:       Append  $d_i$  to  $D$ 
9:     end for
10:    Initialize empty image array  $I_k$  of size  $224 \times 224 \times 3$ 
11:    for  $p = 1$  to  $\text{length}(D)/3$  do
12:      Map triple  $(d_p, d_{p+1}, d_{p+2})$  to RGB pixel  $I_k[p] = (d_p, d_{p+1}, d_{p+2})$  ▷ Equation (6)
13:    end for
14:    Resize or pad  $I_k$  to  $224 \times 224$  if necessary
15:    Save  $I_k$  as PNG image
16:  end for
17:  return  $X_i^B = \{I_1, I_2, \dots, I_8\}$ 

```

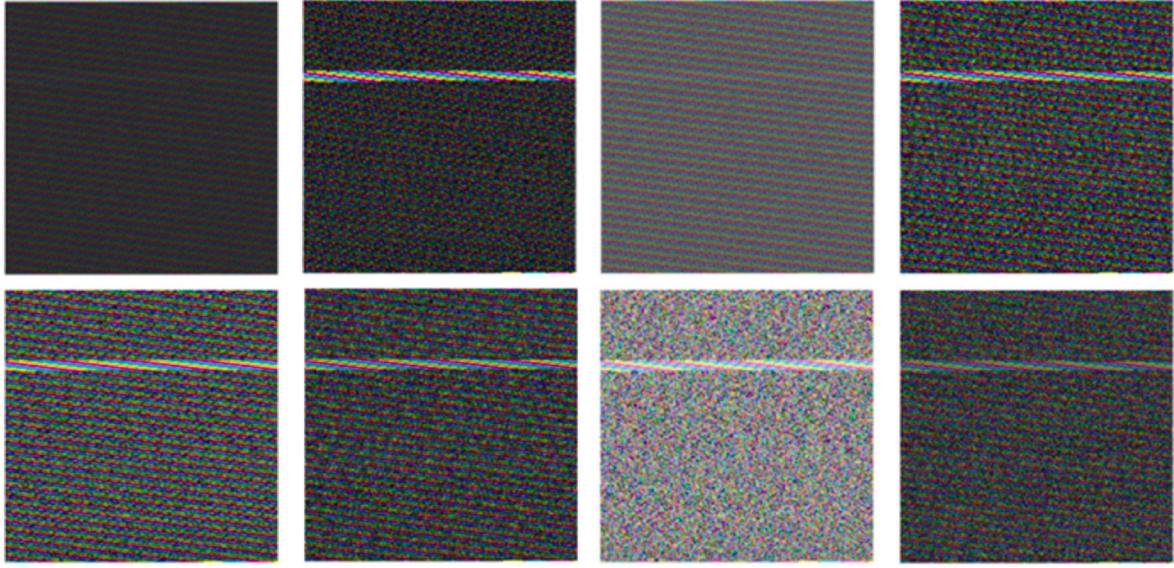


Fig. 4. The example of Id: LOP6HaJKXpkic5dyuVnT Convert 8 color images. (For interpretation of the references to color in this figure legend, the reader is referred to the Web version of this article.)

$$\mathbf{b}^{(k)} = \mathbf{b}_{k+1} \mathbf{b}_{k+2} \dots \quad (15)$$

For each binary string, we generate 8 slightly varied versions, creating a unique image for each one. This approach is particularly beneficial for data that may contain corrupted sections. Through this fine-tuning strategy, we can somewhat compensate for data incompleteness, ensuring the visual features of the malware are more comprehensive and richer.

Ultimately, we save this RGB data as 224×224 PNG images. Consequently, each *byte* file is transformed into 8 such images, with each image capturing features from a different perspective of the original data. Fig. 4 shows the Id as LOP6HaJKXpkic5dyuVnT converted into 8 color maps.

3.3.3. Bilinear attention network

Bilinear Attention Network (BAN), initially introduced by Kim et al. (2018), is a widely adopted approach for modeling high-order interactions between two modalities. In our work, we adapt BAN to combine byte and ASM features, which are naturally distinct in structure and semantics. Additionally, transfer learning (Xiang et al., 2010), (Zhuang et al., 2020) enables leveraging pretrained ViT and VGG16 models, avoiding extensive training while capturing representative features. Our model builds on these works by integrating BAN with a novel 8-view malware representation pipeline. It can learn bilinear interactions between two different modalities of data, resulting in a more effective joint representation. In this malware classification task, we can consider ASM and *byte* files as two different modal data. ASM files contain structured information about the malware instructions, while *byte* files contain raw information about the malware bytecode. BAN mainly consists of two parts:

The first is the bilinear attention layer: This layer is used to calculate the bilinear interaction for each channel between the two-modal data.

Previously, we converted the ASM and *byte* files in the BIG2015 dataset into multi-channel feature input, $\mathbf{X} \in \mathbb{R}^{N \times \rho}$ and $\mathbf{Y} \in \mathbb{R}^{M \times \phi}$. In order to reduce the input channels of both while retaining key information. We introduce bilinear attention maps $\mathbf{A}$, $\mathbf{A} \in \mathbb{R}^{\rho \times \phi}$, The k th intermediate feature can be expressed by the formula.

$$\mathbf{f}_k = (\mathbf{X}^T \mathbf{U})_k^T \mathbf{A} (\mathbf{Y}^T \mathbf{V}) \quad (16)$$

The theoretical basis is supported by bilinear pooling for cross-modal learning as established in (Kim et al., 2018). Among them, $\mathbf{U} \in \mathbb{R}^{N \times K}$, $\mathbf{V} \in \mathbb{R}^{M \times K}$, are learnable weight matrices, which respectively represent the weights mapped from ASM and *byte* file features to the intermediate representation space, and k is the dimension of the intermediate feature.

$$\mathbf{f}_k = \sum_{i=1}^{\rho} \sum_{j=1}^{\phi} \mathbf{A}_{ij} (\mathbf{X}_i^T \mathbf{U}_k) (\mathbf{V}_k^T \mathbf{Y}_j) = \sum_{i=1}^{\rho} \sum_{j=1}^{\phi} \mathbf{A}_{ij} \mathbf{X}_i^T (\mathbf{U}_k^T \mathbf{V}_k^T) \mathbf{Y}_j \quad (17)$$

$\mathbf{X}_i$, $\mathbf{Y}_j$ which represent respectively the i -th and j -th channels of input attention weights between a pair of feature channels. These weighted feature representations are then integrated through a linear transformation $\mathbf{P}$ to obtain the final feature representation $\mathbf{f}$.

$$\mathbf{f} = \mathbf{P}^T \mathbf{f}_0 \quad (18)$$

Besides, to obtain the attention map $\mathbf{A}$, we use the following formula, which contains a learnable support $\mathbf{p}$ and a scaling factor σ .

$$\mathbf{A} = \text{softmax}_{\sigma} \left(\frac{1}{\sigma} (\mathbf{1} \cdot \mathbf{p}^T) (\mathbf{X}^T \mathbf{U}_0) (\mathbf{V}_0^T \mathbf{Y}) \right) \quad (19)$$

This attention map $\mathbf{A}$ not only captures the interaction between channels, but is also normalized by the *softmax* function, ensuring that the attention weights of each channel sum to 1.

The second is low-rank bilinear pooling: this layer is used to achieve a joint representation of each channel pair.

In order to further integrate the joint representation of each channel pair, this layer adopts a residual learning strategy multi-modal residual

Table 2

The summary of Big2015 malware data.

Malware Family	Number
Ramnit	1541
Lollipop	2478
Kelihos_ver3	2942
Vundo	475
Simda	42
Tracur	751
Kelihos_ver1	398
Obfuscator.ACY	1228
Gatak	1013

network MRN, MRN integrates the joint representation from multiple bilinear attention maps, and the output of the $(i+1)$ th time is expressed as

$$\mathbf{f}_{i+1} = \mathbf{BAN}_i(\mathbf{f}_i, \mathbf{Y}; \mathbf{A}_i) \cdot \mathbf{1}^T + \mathbf{f}_i \quad (20)$$

Here $\mathbf{f}_{i+1}$ contains information from the previous stage and new features calculated through the bilinear attention mechanism. In this way, the flow of information in the model is maintained, which helps gradients propagate in deep networks, thereby alleviating the vanishing gradient problem.

Unlike the key-query-value structure in self-attention (Zhou et al., 2024), BAN directly models bilinear interactions $\mathbf{v}_i^T \mathbf{W} \mathbf{u}_j$ between two feature sets $\mathbf{v}, \mathbf{u}$ from different modalities. This structure captures second-order dependencies and allows richer fusion compared to linear attention mechanisms. Moreover, BAN emphasizes *cross-channel attention* instead of sequential alignment used in transformer models.

Finally, in order to obtain the feature representation for classification, we sum the channel dimensions of the last output f_G , which provides a rich feature representation that integrates all modal information for subsequent classifiers. This representation is able to capture subtle differences and complex interactions between ASM and byte file characteristics, thereby improving malware identification accuracy. Through this fine-tuning strategy, we can to some extent compensate for data incompleteness, ensuring a more comprehensive and rich visual representation of the characteristics of malware.

3.3.4. Theoretical basis of transfer learning

In our malware classification framework, transfer learning enables effective feature extraction from limited data by leveraging models pretrained on large-scale natural image datasets. Let $\mathcal{D}_S$ and $\mathcal{D}_T$ denote the source and target domains, respectively, where $\mathcal{D}_S$ corresponds to natural images and $\mathcal{D}_T$ to malware RGB images generated from byte and ASM files. We use pretrained VGG16 and Vision Transformer (ViT) models as fixed feature extractors for byte- and ASM-based images, respectively, assuming that low-level visual features (e.g., edges, shapes) are transferable across domains.

Given a frozen source model f_S and a trainable target classifier f_T , the learning objective on the target domain is defined as

$$\mathcal{L}_T(f_T) = \mathbb{E}_{(x,y) \sim \mathcal{D}_T} [\mathcal{L}(f_T(f_S(x)), y)] \quad (21)$$

where x is a malware image and $\mathcal{L}(\cdot)$ denotes the cross-entropy loss. This strategy avoids training from scratch, reduces computational cost, and provides robust feature representations for subsequent multimodal fusion. It also improves generalization in the presence of limited labeled malware samples and facilitates effective integration of byte- and ASM-level information.

Table 3
Implementation details and hyperparameters.

Category	Parameter	Value
Training	Epochs	30
	Batch Size	32
	Optimizer	Adam
	Learning Rate (ViT & VGG16)	2e-6
	Patience: 3	
Model architecture	MLP Input Dimension	1024
	MLP Hidden Layer Size	512
	MLP Number of Dense Layers	2
	MLP Activation	ReLU
	Dropout Rate	0.3
	Output Classes	9

4. Experiments and results

4.1. Malware datasets

The Big2015 dataset is utilized for malware classification in this study, which released by Microsoft on the Kaggle platform (Microsoft Malware Classification Challenge). This dataset is widely used in malware research field. Each sample in the dataset consists of two types of files: ASM files (produced by the disassembly tool IDA) and hexadecimal byte files. This dataset includes a variety of malware, spanning virus families like Ramnit, Kelihos ver.3, Simda, and Kelihos ver.1, as well as trojans such as Vundo, Tracur, and Gatak. Additionally, it features adware like Lollipop and complex malware such as Obfuscator. ACY. The dataset was divided, with a 4:1 ratio for cross-validation and testing. As detailed in Table 2, the detailed statistics on the number of each malware family in the BIG2015 dataset were provided. In training process, only the training data was utilized, incorporating stratified five-fold cross-validation. This methodology involves dividing the dataset into five segments, utilizing one for validation and the remainder for training, iteratively for five cycles, each with a distinct validation segment. Such an approach aids in evaluating model performance across different data subsets, thereby offering a more accurate estimation of generalization ability.

4.2. Implementation details

All experiments were conducted on an NVIDIA A100 GPU from Alibaba Cloud to ensure efficient training and reproducibility. To comprehensively evaluate the effectiveness of our proposed model, we compared it against several competitive baseline methods, including both classical and state-of-the-art deep learning architectures. These baselines include ResNet, Vision Transformer, Swin-Transformer, VGG16, as well as recently published models such as FACILE (Zou et al., 2024), SDIF-CNN (Kumar and Panda, 2023), and MobileNet (Palma Salas et al., 2023).

The fused features are utilized for classification employing multi-layer perceptron (MLP). The parameters of the MLP model are detailed in Table 3.

In this study, we employed GridSearchCV to find the optimal hyperparameter combination. GridSearchCV employs an exhaustive search within a predefined hyperparameter space to effectively identify the best parameter combination. The process commenced with the definition of a hyperparameter space and the determination of the range of potential values for each parameter. Subsequently, a training and validation set was established. To ensure reliability and rigor, we integrated grid search with cross-validation. This involved iterating through all combinations within the hyperparameter space, computing the average score for each parameter set. The combination yielding the highest score was identified as the optimal hyperparameter set for the model.

In the parameter tuning, we assessed various hyperparameters

Table 4
Comparison with competing methods (Bold: best performance).

Models	Accuracy	Precision	Recall	F1-score	Time (ms)
Resnet	0.8424	0.8401	0.8403	0.8402	20
Vision Transformer	0.8410	0.8415	0.8412	0.8413	45
Swin-Transformer	0.8522	0.8623	0.8522	0.8572	55
VGG16	0.8945	0.9056	0.8945	0.9000	40
FACILE (Zou et al., 2024)	0.9720	0.9398	0.9213	0.9304	32
SDIF-CNN (Kumar and Panda, 2023)	0.9658	0.9689	0.9390	0.9537	48
MobileNet (Palma Salas et al., 2023)	0.9841	0.9444	0.9825	0.9631	37
The proposed method	0.9878	0.9879	0.9874	0.9876	58

including learning rates for Vision Transformer and VGG16 models, MLP classifier hidden layer dimensions, and Dropout rates. Initially, we explored a wide range of values and then narrowed down the focus to identify optimal settings. The Adam optimizer was chosen for weight updates, leveraging its advantages of momentum and adaptive learning rates to facilitate faster convergence and improved performance. We conducted experiments over 30 epochs, a value determined empirically using five-fold cross-validation and GridSearchCV to identify an appropriate trade-off between training time and generalization. Although deeper models sometimes benefit from longer training (i.e., more epochs), preliminary tests showed that performance gains plateaued after ~ 25 epochs while longer training introduced minor overfitting. The dropout rate (0.3) was similarly selected based on its ability to reduce validation loss without underfitting, aligned with prior studies in malware classification using image representations (Zou et al., 2024; Kumar and Panda, 2023).

Given that this is a multi-class classification task with nine malware families, we use categorical cross-entropy as the loss function (Janocha and Czarnecki, 2017). It is defined as

$$\mathcal{L}_{CCE} = - \sum_{i=1}^C y_i \log(\hat{y}_i) \quad (22)$$

where C is the number of classes, $y_i \in \{0, 1\}$ is the ground truth one-hot label, and $\hat{y}_i \in [0, 1]$ is the predicted probability for class i . This loss penalizes incorrect predictions in proportion to the negative log-likelihood of the true class, effectively guiding the model to output high confidence for the correct class.

Through a series of hyperparameter optimizations and training strategies, the performance of the proposed model for malware detection tasks was enhanced. Details on the hyperparameter settings and experimental results will be provided in subsequent sections. Table 3 summarizes the hyperparameters used in the experiments.

4.3. Evaluation metrics

This study adopts accuracy, precision, recall, and F1-score as evaluation metrics to assess the performance of the proposed method. These metrics, validated by prior research, are highly indicative of the effectiveness of malware classification. Together, they constitute a comprehensive evaluation framework that reflects the ability to identify and differentiate between nine categories of malware. Accuracy measures

Table 5

Ablation experimental results (Bold: best performance).

Models	Accuracy	Precision	Recall	F1-score
Model-1	0.9378	0.9379	0.9178	0.9278
Model-2	0.9778	0.9789	0.9790	0.9789
Model-3	0.9454	0.9456	0.9454	0.9455
Model-4	0.9687	0.9695	0.9687	0.9691
The proposed method	0.9878	0.9879	0.9874	0.9876

the ratio of correctly predicted samples to the total number of samples, defined as $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$. Precision calculates the proportion of actual positive samples among those predicted as positive: $Precision = \frac{TP}{TP+FP}$. Recall represents the proportion of samples correctly predicted as positive out of all actual positive samples, $Recall = \frac{TP}{TP+FN}$. The F1-score is the harmonic mean of precision and recall, serving as a comprehensive metric that considers both: $F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$. Here, TP , TN , FP , and FN represent true positives, true negatives, false positives, and false negatives, respectively.

4.4. Results

4.4.1. Comparison with competing methods

To comprehensively assess the malware classification performance of the proposed model, the proposed method compared it with the recent state-of-the-art malware classification models and the classic deep learning models. All models were executed with similar hyperparameter settings. The results are detailed in Table 4. Notably, except for the FACILE and the proposed models, the others did not use cross-validation. The study found that models trained with cross-validation did not achieve the same performance level initially reported. Therefore, the proposed method comparison strategy ensures fairness and credibility in the evaluation. Importantly, under the same training environment, our model outperforms other models in all evaluation metrics, with accuracy, precision, recall, and F1-score reaching 0.9878, 0.9879, 0.9874, and 0.9876, respectively. The proposed method makes full use of the Big2015 dataset, analyzing not just *byte* files but also learning from ASM file information. In dealing with *byte* files, the proposed method explored multi-image fusion strategies. By employing a model based on VGG16 and incorporating attention mechanisms. The proposed model is able to more effectively understand the connections

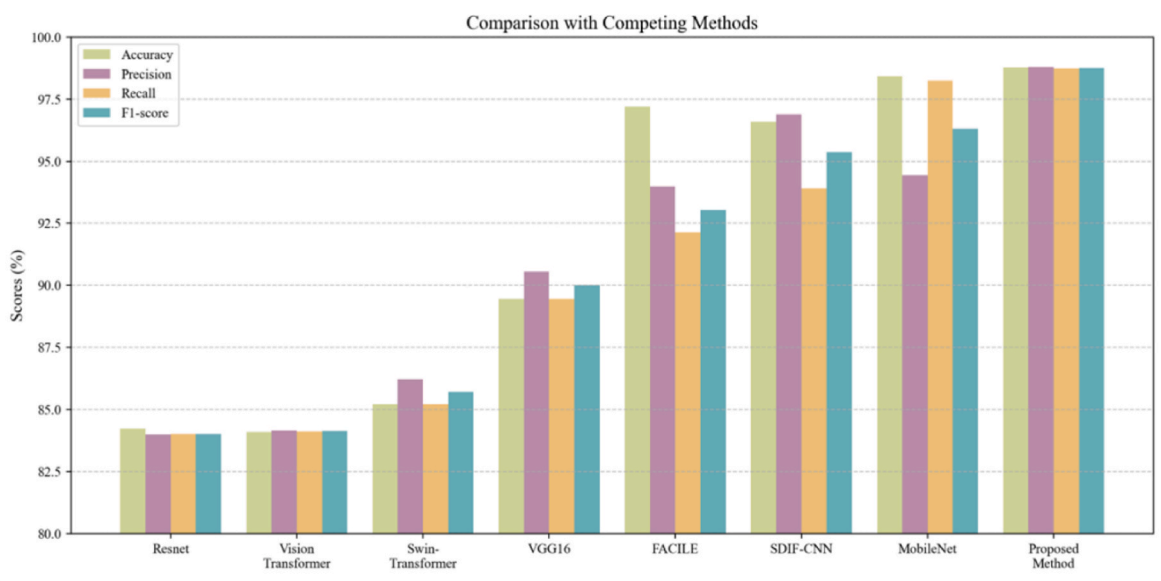


Fig. 5. The comparison of malware classification models.

Table 6

Performance comparison of image normalization strategies (Bold: best performance).

Strategy	Accuracy	Precision	Recall	F1-Score
Zero-Padding (224×224)	0.9878	0.9879	0.9874	0.9876
Resizing (224×224)	0.9652	0.9648	0.9635	0.9641
Reduced Dim (128×128)	0.9515	0.9502	0.9498	0.9500
Reduced Dim (64×64)	0.9380	0.9365	0.9358	0.9361

and key features between different *byte* images (i.e., different representations of the same malware). Furthermore, we used a blended attention network to merge ASM and *byte* file information, learning both local and global features, thus achieving high values across all metrics. Hence, the exemplary performance of the proposed method meets our expectations.

In contrast, most models rely solely on *byte* information from the Big2015 dataset to generate a single-color image for malware family classification, failing to learn all features fully.

As shown in Fig. 5, the proposed method achieves the highest score across all metrics. Compared to MobileNet and SDIF-CNN, it improves F1-score by 2.45% and 3.39%, respectively, validating the effectiveness of multi-view and BAN integration.

To better understand the practical applicability of our model, we compare the inference time required for malware classification across various existing methods. Inference time is a critical factor in real-world deployment, particularly in time-sensitive cybersecurity scenarios such as endpoint detection and response. Despite employing a dual-branch architecture using VGG16 and ViT for feature extraction and Bilinear Attention Network (BAN) for fusion, our model maintains competitive inference efficiency. This is primarily due to the use of pre-trained feature extractors without fine-tuning during inference and a light-weight BAN structure. As shown in Table 4, the proposed method achieves a per-sample inference time of approximately 58 ms-only slightly above that of SDIF-CNN and FACILE, yet delivering significantly higher classification accuracy.

4.4.2. Ablation study

In this research, we explore the significance of each component, confirming their essential role in the model. Multiple experiments were conducted to assess the impact of every innovative element in the proposed model across two datasets, as follows: (1) Model-1 utilized a single image, replacing the module that generates eight images in the proposed method. (2) Model-2 eliminated ASM features. (3) Model-3 omitted *byte* features. (4) Model-4 discarded the BAN block, opting for standard concatenate for feature fusion. Importantly, all other settings and parameters were kept consistent with the original method. Table 5 outlines the performance of models under different configurations.

Results in Table 5 indicate a roughly 5% reduction in all metrics for Model-1 without the 8-image generation, validating our data enhancement strategy. Model 2 showed a 1% decline without ASM features, indicating the global features' influence on model efficacy. Model-3, lacking *byte* features, saw a 3% decrease in accuracy, underscoring the importance of *byte* features in representing malware attributes

Table 7

The effect of different images on the results (Bold: best performance).

Num	Accuracy	Precision	Recall	F1-score
1	0.9378	0.9379	0.9178	0.9278
2	0.9475	0.9472	0.9275	0.9372
3	0.9488	0.9489	0.9290	0.9388
4	0.9595	0.9594	0.9392	0.9492
5	0.9503	0.9501	0.9303	0.9401
6	0.9610	0.9615	0.9412	0.9512
7	0.9722	0.9723	0.9524	0.9622
8	0.9878	0.9879	0.9874	0.9876

Table 8

Comprehensive evaluation (Bold: best performance).

Features	model	Accuracy	Precision	Recall	F1-score
Byte	VGG16+concat	0.9754	0.9756	0.9754	0.9755
	VGG16 + average	0.9658	0.9660	0.9658	0.9659
	VGG16+set pooling	0.9585	0.9586	0.9585	0.9585
	VGG16+sum	0.9625	0.9635	0.9625	0.9630
Byte	VGG16 + concatenate	0.9754	0.9756	0.9754	0.9755
	VGG19 + concatenate	0.9685	0.9688	0.9685	0.9686
	ViT + concatenate	0.9618	0.9629	0.9618	0.9623
	Swin-transformer + concatenate	0.9581	0.9600	0.9581	0.9590
	Resnet + concatenate	0.9629	0.9638	0.9629	0.9633
	CNN + concatenate	0.9654	0.9656	0.9653	0.9654
ASM	ViT	0.9454	0.9456	0.9454	0.9455
	VGG16	0.9352	0.9325	0.9265	0.9295
	VGG19	0.9232	0.9223	0.9245	0.9234
	RESNET	0.9025	0.9054	0.9045	0.9049
	SWIN-transformer	0.9388	0.9387	0.9388	0.9387
Byte + ASM	CNN	0.9055	0.9058	0.9055	0.9056
	VGG16+ViT + BAN	0.9771	0.9773	0.9771	0.9772
	VGG16+ViT + concatenate	0.9677	0.9687	0.9677	0.9682
	VGG16+ViT + set pooling	0.9554	0.9554	0.9555	0.9554
	The proposed method	0.9878	0.9879	0.9874	0.9876

comprehensively. Model-4, which bypassed BAN for fusion, also experienced a 2% drop in metrics. A comparison of each component with the proposed method revealed that the model only reaches satisfactory performance when all proposed innovative elements are integrated.

The ablation studies in Table 5 were conducted to systematically evaluate the impact of each core component in our architecture. To this end, we designed four model variants. The first variant, Model-1, replaces our multi-view image strategy with a single image per malware sample to assess the effect of view diversity. Model-2 excludes ASM features entirely, relying only on byte-level inputs processed through VGG16, while Model-3 removes byte features and uses only ASM-based inputs processed by the ViT model. Model-4 retains both input modalities but replaces the bilinear attention mechanism with simple feature concatenation for fusion. All variants were trained using the same hyperparameters and data splits as the full model to ensure that any observed performance differences can be attributed solely to architectural changes.

Furthermore, to validate the effectiveness of our data preprocessing strategy, we conducted an additional ablation study focusing on image normalization techniques. Specifically, we evaluated the impact of zero-padding (our method) versus image resizing and dimensional reduction strategies. We compared four settings: (1) Zero-Padding: preserving the original byte sequence and padding with zeros to 224×224 ; (2) Resizing: using bicubic interpolation to force the binary to 224×224 ; (3) Reduced Dim (128×128): down-sampling to 128×128 before up-sampling; and (4) Reduced Dim (64×64). As shown in Table 6, the proposed zero-padding strategy significantly outperforms all resizing variants. The direct resizing approach resulted in a 2.26% drop in accuracy compared to zero-padding, while further reducing dimensions to 128×128 and 64×64 led to even greater performance degradation (accuracy dropping to 0.9515 and 0.938, respectively). These results confirm that interpolation-based resizing introduces noise that blurs the discrete opcode features, whereas zero-padding maintains the structural integrity of the malware binaries, allowing the model to extract more accurate features.

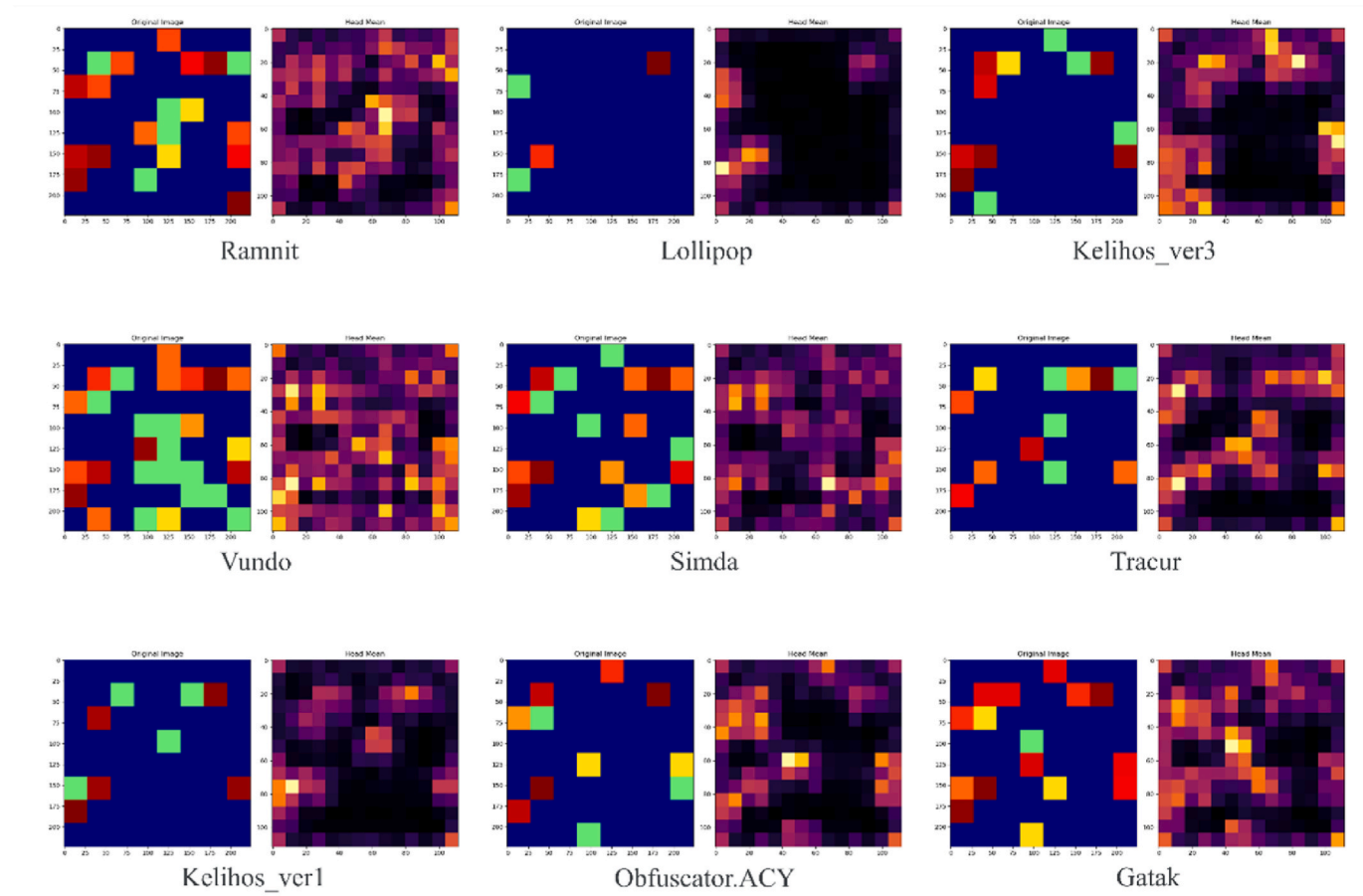


Fig. 6. The visualizing ASM image feature of malware families.

5. Discussion

5.1. Number of generated images

In this study, we conduct the strategy that each malware binary sequence generates 8 images. This raises the question of whether variations in the number of generated images could influence model performance. To investigate this possibility, this work generated 1 to 8 images for each malware binary sequence. The results are stated in Table 7. For a single image, we followed the same approach as others. For two images, we shifted the sequence by one to generate, and so on, up to 8 images., averaging five runs, covering four key metrics. Employing a fixed sequence of eight images yielded the optimal results in the tests. That means in our setup, more images help the model do better.

5.2. Evaluating feature fusion strategies

A series of comparative experiments were conducted in this study to assess efficacy of the proposed architecture, with a particular focus on

evaluating diverse models' ability to process *byte* and ASM image features. It also delved into the advantages of different feature fusion methods and the role of BAN (Bilinear Attention Networks) in integrating these features. All experimental procedures employed 5-fold cross-validation, averaging the outcomes of the test sets across five iterations.

As depicted in Table 8, an evaluation of four feature fusion strategies (concatenate, average, set pooling, and sum) applied to the amalgamation of features from 8 images, revealed concatenation as the superior method in all metrics: accuracy, precision, recall, and F1-score, recording respective scores of 0.9754, 0.9756, 0.9754, and 0.9755. The dominance stems from its capacity to retain comprehensive feature information, unlike the average, sum, and set pooling methods, which may compromise some data during the fusion process.

A comparative study underscored the distinct superiority of VGG16 in *byte* image feature extraction relative to competing models, especially with the implementation of a concatenate fusion approach. This could be due to its deep feature extraction capability and sensitivity to detail. For ASM image feature extraction, the ViT model outperformed other models, achieving high scores in accuracy, precision, recall, and F1-Score.

While our method incorporates pre-trained models like VGG16 and ViT, the core innovation stems from the novel multi-view input generation strategy, and the BAN-based fusion framework, which jointly exploits both local byte-level and global ASM-level representations. As shown in our comparative evaluations, this integration enables more effective feature interactions than previous methods (e.g., direct concatenation (Zou et al., 2024), or unimodal approaches (Kumar and Panda, 2023)). Thus, our contribution lies not merely in model reuse but in architectural composition and modality design, achieving superior

Table 9
Computational performance comparison (with transfer learning).

Method	FLOPs (G)	Training Time (h)	Memory (GB)	Inference Time (ms)
FACILE	8.5	14.0	12.3	32
SDIF-CNN	12.3	21.7	18.7	48
MobileNet	3.8	11.7	7.2	37
Our Method	8.9	15.6	45.2	58

results across all metrics. Integrating the attention mechanism into the VGG16 transfer learning framework underwent a thorough assessment via an in-depth evaluation of the ultimate model. Across five iterations of the experiment, the mean results showcased that all metrics reached an impressive accuracy of 0.9878, affirming the efficacy and highlighting the enhanced performance attributed to the attention mechanism within the transfer learning framework.

5.3. Interpretability

The proposed model demonstrates remarkable performance in differentiating malware families, yet the clarity of its decision-making process requires further enhancement. Integrating various models with a comprehensive approach to malware attributes complicates understanding the decision-making processes. Utilizing eight bytecode-derived images as inputs poses challenges in identifying pertinent features. Focus centers on visualizing ASM file features, with the assumption that they disclose vital visual cues once processed through the BAN. Notably, we delve into the attention distribution of the model in the ASM view to identify key features that are critical for malware family classification. The visual representation of ASM file attributes associated with malware families (Fig. 6) shows that ViT focuses on prominent color regions, suggesting its ability to identify information-rich regions for accurate classification. These findings emphasize the capability of ViT in identifying key features and segmenting malicious code, thus improving the interpretability of its decisions.

5.4. Computational complexity and practical considerations

Our model integrates VGG16, ViT, and BAN, which increases structural complexity. However, both VGG16 and ViT are used in frozen pre-trained form, so only forward passes are performed during training. This greatly reduces memory and computation. BAN, which introduces bilinear interactions, is implemented using low-rank approximation (rank $k = 32$), lowering its complexity from $\mathcal{O}(d^2)$ to $\mathcal{O}(dk)$. Overall, over 95% of model parameters are fixed, with only about 2.4 million parameters trained out of 49 million. This design ensures our model remains computationally efficient while retaining strong performance.

Table 9 presents the computational performance of our model compared with other representative baselines. While our model has higher inference memory usage due to multi-view processing, the training time is comparable to other multi-component architectures like

FACILE and SDIF-CNN. Notably, the reliance on frozen backbones allows efficient training without the need for full end-to-end optimization of deep networks.

Furthermore, a series of efficiency-oriented techniques were adopted, including dynamic multi-view image generation (which reduced storage demands by nearly 90%), mixed precision training (which reduced memory usage by $\sim 30\%$), and inference optimization via TensorRT, which decreased latency with negligible accuracy loss.

Overall, while our model architecture is more complicated, its training cost is well-controlled, and its deployment feasibility is ensured by the high throughput, accuracy, and optimization strategies in place.

5.5. Limitation and future work

Although we utilized one of the most comprehensive publicly available malware datasets-BIG2015-which includes both binary byte files and corresponding assembly (ASM) files, we acknowledge certain limitations in terms of dataset diversity and platform coverage. Most existing malware datasets only provide either raw byte files or grayscale image representations, but not both modalities, which restricts the direct applicability of our dual-modality approach. Furthermore, BIG2015 focuses solely on Windows-based malware and does not include samples from other platforms such as Android, Linux, or iOS, limiting the generalizability of our findings across different operating systems.

In future work, we plan to extend our evaluation to multi-platform malware datasets and investigate whether our model architecture can be adapted to accommodate other binary formats. We are also interested in exploring the optimal number and type of image representations required to effectively capture malware characteristics, as our current design uses eight images per sample. Additionally, we intend to investigate frequency-domain analysis. Recent findings suggest that processing malware images in the JPEG frequency domain using lightweight Transformers can increase detection speed by over 1000% while maintaining high accuracy, offering a promising direction for real-time mobile deployment (Zou et al., 2025). Due to the scarcity of multi-modal datasets across diverse platforms, we intend to collaborate with security research communities or construct synthetic datasets where feasible, to evaluate cross-platform generalization and enhance model robustness.

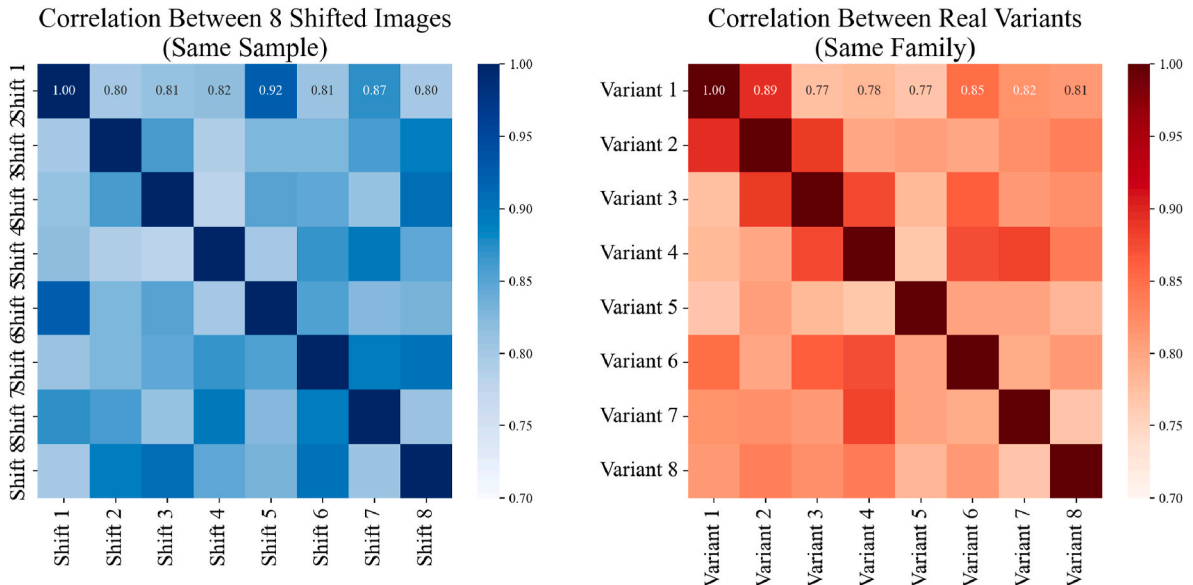


Fig. 7. Correlation matrices comparing bit-shifted images and real malware variants.

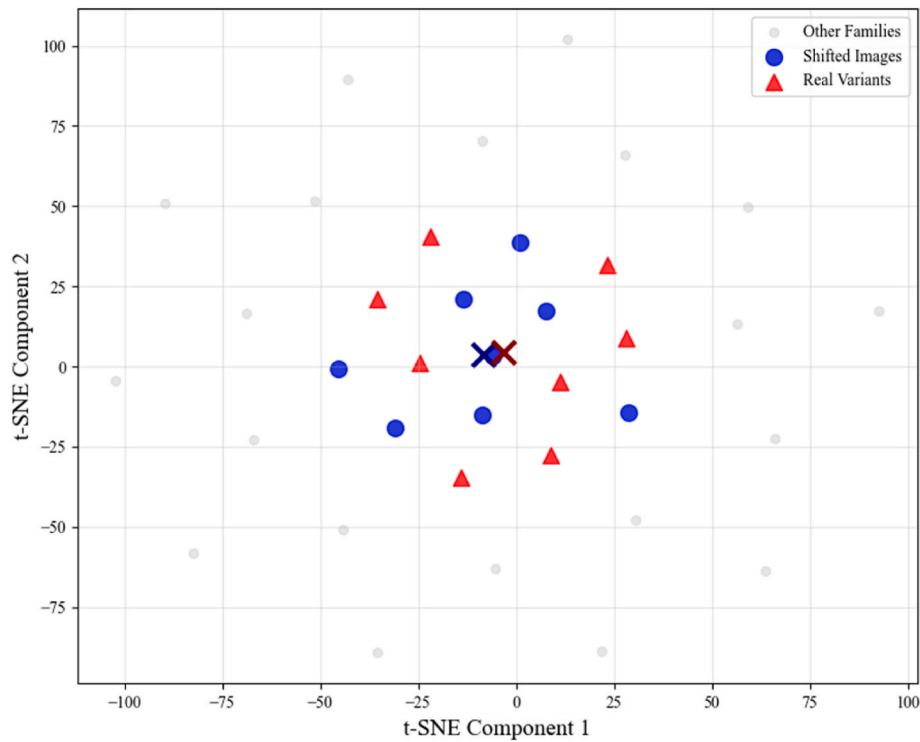


Fig. 8. The t-SNE visualization of real variant samples form compact clusters with similar intra-cluster variance and centroid spacing.

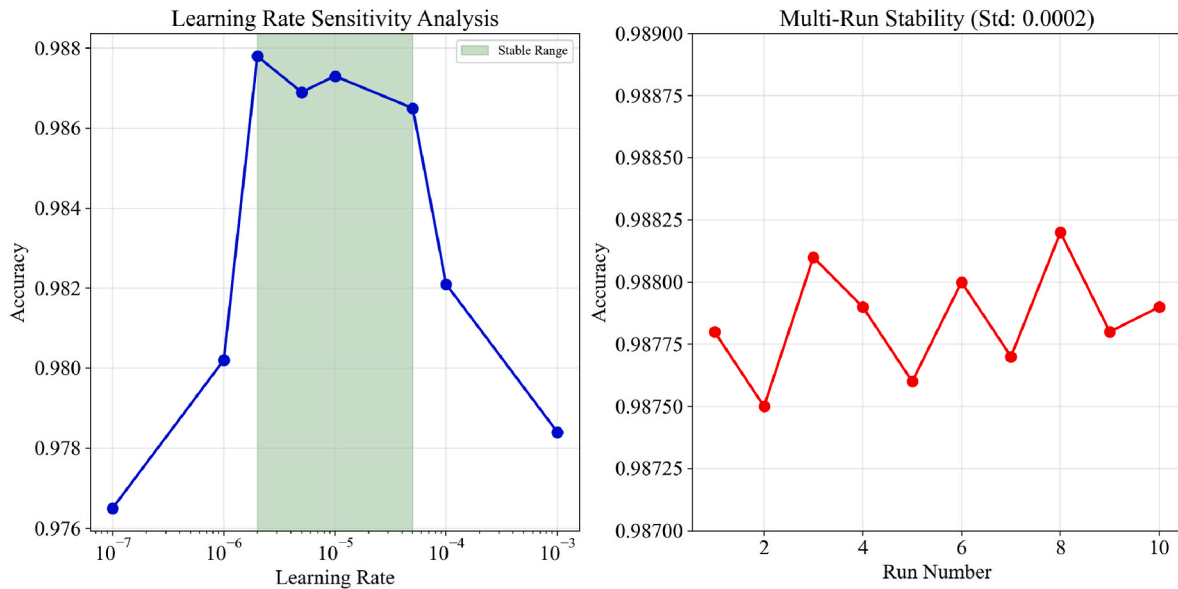


Fig. 9. The illustration of stability analysis results.

5.6. Relationship between generated images and malware variants

To investigate the relationship between the generated bit-shifted images and actual malware variants, we conducted an extensive analysis across multiple malware families. Authentic variant samples were collected from the same families as those in the BIG2015 dataset, sourced from Virus Total and verified using multiple antivirus engines.

For each malware family, we computed Pearson correlation coefficients to compare two types of relationships: one among the eight bit-shifted images derived from a single malware sample, and the other among authentic variant samples within the same family. As shown in Fig. 7, the resulting correlation matrices indicate that both groups

exhibit similar patterns of high intra-group correlation, generally ranging between 0.75 and 0.95, with comparable variance structures. These findings suggest that our bit-shifting approach effectively preserves intra-family similarity patterns and captures structural variations similar to those observed in real malware evolution.

Beyond pixel-level correlation, we further analyzed the distribution of features in high-dimensional space using t-SNE. We extracted 512-dimensional features from both bit-shifted images and real variants via the conv5_3 layer of a pre-trained VGG16 model. After reducing dimensions to 128 using PCA, we projected the features into 2D using t-SNE for visualization.

As shown in Fig. 8, both bit-shifted and real variant samples form

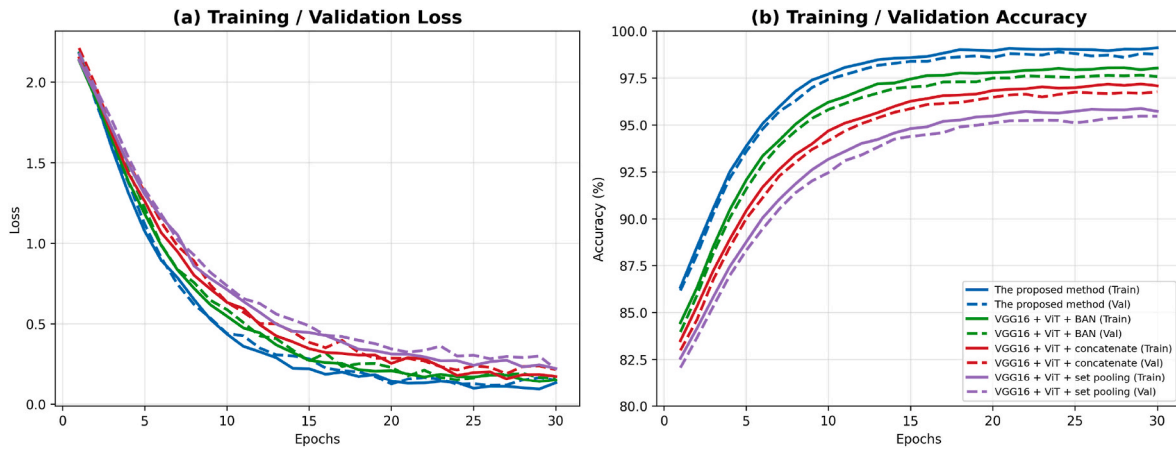


Fig. 10. Training and validation convergence of the model.

Table 10
Per-family classification results.

Malware Family	Precision	Recall	F1-Score	Samples	Difficulty Factors
Ramnit	0.993	0.991	0.992	1541	Low variability
Lollipop	0.989	0.987	0.988	2478	Large dataset
Kelihos_ver3	0.986	0.984	0.985	2942	Clear patterns
Vundo	0.953	0.949	0.951	475	High variant diversity
Simda	0.925	0.921	0.923	42	Limited samples
Tracur	0.968	0.965	0.966	751	Moderate complexity
Kelihos_ver1	0.974	0.97	0.972	398	Similar to ver3
Obfuscator. ACY	0.942	0.938	0.94	1228	Obfuscation techniques
Gatak	0.961	0.959	0.96	1013	Moderate patterns

compact clusters with similar intra-cluster variance and centroid spacing. Approximately 90% of each group's samples lie within 1.5 standard deviations of their cluster centers. The silhouette scores (0.742 vs. 0.729) and a Mann-Whitney U test ($p = 0.23$) confirm no significant difference in cluster structure, supporting the similarity between generated images and real malware variants. These results collectively demonstrate that our bit-shifting strategy does not introduce artificial distortions, but rather captures semantically meaningful variations that closely mirror the characteristics of actual malware variants.

5.7. Discussion on hardware-dependent variability

All experiments in this study were conducted under a consistent hardware environment, using an NVIDIA A100 Tensor Core GPU (80G), Intel Core i9 CPU, and 128 GB RAM, with PyTorch 2.0 as the deep learning framework. To ensure fair comparison, all baseline models were re-trained under the same settings, including batch size, input image size, and number of training epochs. While some original studies may have used different hardware configurations, our unified setup ensures that performance differences are attributable to model design rather than environmental factors. This setup enhances the fairness, transparency, and reproducibility of our experimental results.

5.8. Parameter sensitivity and stability analysis

We systematically varied critical hyperparameters and monitored performance stability, as demonstrated in Fig. 9. **Learning Rate Sensitivity:** We tested learning rates from $1e-7$ to $1e-3$ and observed that performance remains stable within $[2e-6, 5e-5]$, showing graceful degradation rather than oscillatory behavior outside this range. **Multi-**

Run Stability: Across 10 independent runs with different random seeds, our method demonstrates remarkable consistency (standard deviation $< 0.3\%$ for accuracy), confirming stable convergence without periodic fluctuations.

To further analyze the efficiency and accuracy of the proposed architecture, we conducted an empirical convergence study focusing on the BAN and ViT components under the transfer learning setting. Fig. 10 illustrates the training and validation loss and accuracy curves over 30 epochs on the BIG2015 dataset. As observed, the full model incorporating BAN and ViT exhibits smooth and stable convergence, with the validation accuracy reaching a clear plateau after approximately 25 epochs. Compared with the variant using simple feature concatenation, the BAN-based fusion converges faster and achieves higher final validation accuracy, indicating that BAN not only enhances cross-modal feature interaction but also improves optimization efficiency.

Furthermore, the ViT-based ASM branch demonstrates consistent convergence behavior without noticeable oscillation or divergence, despite operating on opcode-derived color images that differ substantially from the natural images used in pretraining. This empirical observation confirms that transfer learning from the source domain to the target malware domain converges reliably, and that using a frozen ViT backbone provides a stable optimization landscape for malware representation learning.

To evaluate robustness, we repeated the training process ten times with different random seeds. Across these runs, the standard deviation of validation accuracy remained below 0.3%, further confirming the convergence stability and reproducibility of the proposed model.

5.9. Per-family classification analysis

To assess the effectiveness of our method across different malware families, we conducted a detailed per-family evaluation from three perspectives: classification performance, confusion analysis, and modality-specific contributions.

5.9.1. Family-specific performance

As shown in Table 10, our model achieves strong performance across most malware families. High F1-Score is observed for families such as Ramnit (0.992), Lollipop (0.988), and Kelihos_ver3 (0.985). In contrast, relatively lower performance is seen for Simda (0.923) due to its limited sample size, and Vundo (0.951), which contains a wide range of variants and complex behavior. Importantly, our model is able to successfully distinguish between closely related families such as Kelihos_ver1 and Kelihos_ver3, achieving 0.972 and 0.985 accuracy respectively, demonstrating its capacity for fine-grained classification.

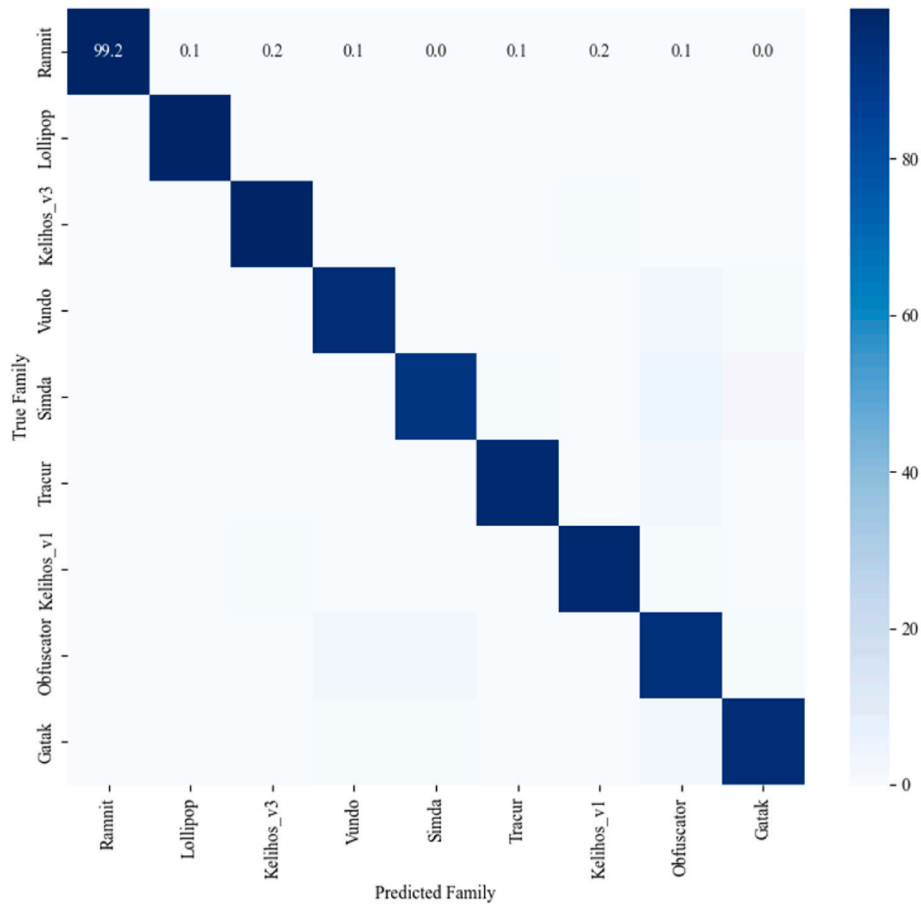


Fig. 11. The confusion matrix of malware family classification.

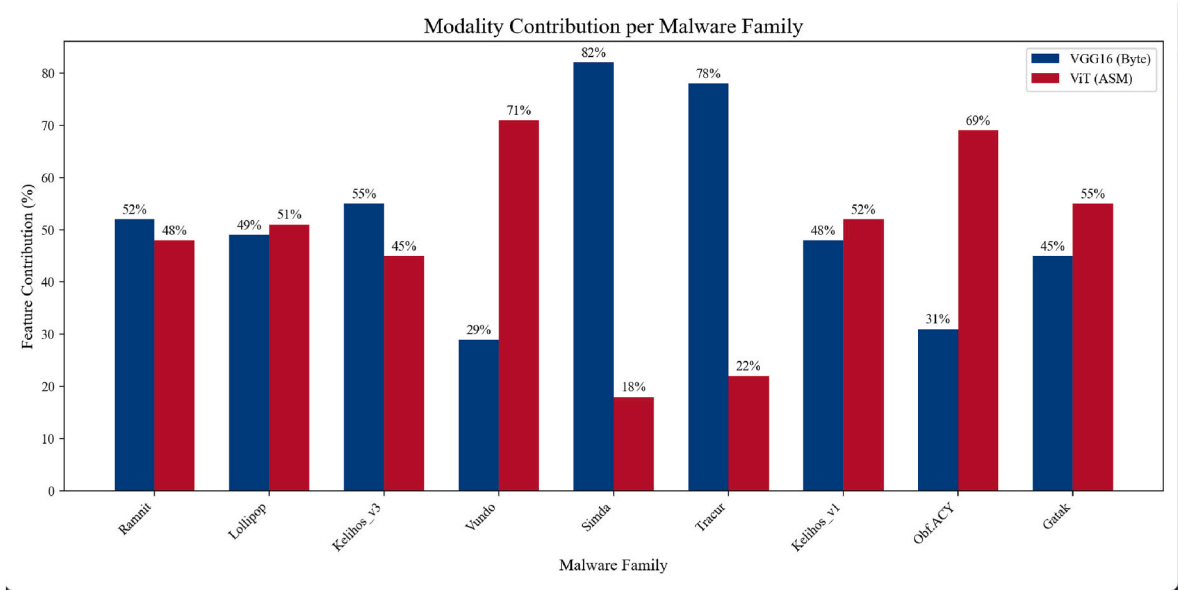


Fig. 12. Bar chart showing VGG16 vs ViT contribution per family.

5.9.2. Cross-family confusion

The confusion matrix in Fig. 11 further illustrates these results. Most misclassifications are concentrated between Obfuscator, ACY and Gatak, with a mutual confusion rate of 2.1%, likely due to similar obfuscation techniques shared by the two families. Kelihos variants, on the other hand, exhibit minimal confusion (only 0.8%), highlighting the strength

of our multi-view design in capturing subtle structural differences.

5.9.3. Feature contribution by family

We also analyzed how different malware families rely on byte-level versus ASM-level features. As visualized in Fig. 12, some families benefit more from byte-based features extracted by VGG16, such as

Table 11

Classification performance under Byte-Alignment Obfuscation (1-4 byte padding).

Sample ID	True Family	Padding	FACILE	SDIF-CNN	MobileNet	Ours
Xj8P ...	Obfuscator.ACY	+1 Byte	✓	✓	✓	✓
6H7q ...	Kelihos_ver3	+1 Byte	✓	×	✓	✓
9k2L ...	Vundo	+2 Bytes	×	✓	✓	✓
B4mN ...	Lollipop	+3 Bytes	×	×	×	✓
Q2zR ...	Gatak	+4 Bytes	×	✓	×	×

Tracur (78% contribution) and Simda (82%), while others rely more on ASM-based features captured by ViT, such as Vundo (71%) and Obfuscator.ACY (69%). Certain families like Ramnit and Lollipop show a balanced dependency on both modalities, indicating the complementary nature of the two representations. These results underscore the necessity of a multi-modal fusion approach for robust and generalizable malware classification.

5.10. Case study: robustness against obfuscated samples

To further validate the superiority of the proposed method in operational cybersecurity scenarios, we conducted an additional empirical test focusing on adversarial byte-padding obfuscation. In real-world cyberattacks, malware authors frequently inject junk bytes or padding data at the beginning of a file to alter its signature and evade static detection. This threat has been underscored by comprehensive surveys recently, which highlight that even minor byte-level perturbations can cause catastrophic misclassification in standard deep learning models (Aryal et al., 2024). Furthermore, automated evasion strategies using evolutionary algorithms and generative models are becoming increasingly prevalent, necessitating robust defense mechanisms like multi-view aggregation (Nesmachnow and Toutouh, 2025). The Impact of Global Phase Shift for deep learning models based on 1D-to-3D image mapping, even a minimal offset (e.g., inserting 1-3 bytes) can be catastrophic. The core vulnerability lies in the RGB Alignment Sensitivity. Since standard techniques typically map three consecutive bytes to a single pixel (b_i, b_{i+1}, b_{i+2}) $\rightarrow$ Pixel $_i$, inserting a non-multiple-of-three offset at the header triggers a Global Phase Shift. This misalignment forces a recombination of byte values across the entire sequence-where a byte originally representing the Red channel may shift to the Green channel-thereby drastically altering the resulting visual texture and effectively destroying the local spatial features relied upon by Convolutional Neural Networks (CNNs). Specifically, we selected five challenging samples from high-variance families and applied random header padding ranging from 1 to 4 bytes. Table 11 details the classification performance compared against three baselines.

The results reveal a clear performance stratification across perturbation levels. Under minor shifts (+1 byte), most models, including the baselines, correctly classified the samples, indicating that modern deep architectures exhibit sufficient translation invariance to tolerate minimal perturbations. As the shift increases to a moderate range (+2/+3 bytes), performance diverges sharply: all baselines fail due to phase-shift-induced texture disruption, whereas our method remains robust, confirming that the 8-view strategy effectively produces alignment-invariant representations that preserve critical visual features. Under extreme perturbation (+4 bytes), all models misclassify, suggesting that severe structural corruption degrades both visual textures and the semantic integrity of the ASM branch, defining the current limit of the dual-level fusion. Overall, these findings corroborate the distinct advantage of our proposed method in mitigating alignment-based evasion attacks, thereby bridging the critical gap between theoretical

robustness guarantees and practical real-world performance.

6. Conclusion

In this study, we propose a novel multi-view-based network for malware classification, integrates vision transformer (ViT) and VGG16 architectures with a bilinear attention network (BAN). This framework shows significant advantages in accuracy, precision, recall, and F1-score on the Big2015 dataset. By generating eight different color images from each malware binary sequence and exploiting the TF of the opcode images, our approach effectively captures both local and global features of the malware. The role of bilinear attention networks in feature fusion proves to be crucial in improving prediction capabilities by integrating byte file front-end details and global ASM file insights. An innovative attention module utilizing a self-attention mechanism improves the ability to identify subtleties in color images, thereby increasing overall performance. The experimental results confirm the advantages of generating multiple color images byte by byte and confirm that ASM files can somewhat reflect the global characteristics of malware. However, the applicability of the model is limited to the Windows platform due to the data focus on Windows malware. Future work will extend the model to cover a wider range of malware and explore the optimal number of images to comprehensively represent malware characteristics.

CRedit authorship contribution statement

Lijie Chu: Writing – original draft, Methodology, Data curation, Conceptualization. **Yiqian Zhang:** Writing – original draft, Methodology, Data curation. **Yuexi Qiu:** Writing – original draft, Methodology, Data curation. **Jinge Xing:** Writing – review & editing, Methodology, Funding acquisition. **Changjian Zhou:** Writing – review & editing, Supervision, Resources, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Basic Research Support Program for Excellent Young Teachers in Heilongjiang Provincial Undergraduate Universities [grant number: YQJH2023202], and 2025 Smart Campus Special Project of Higher Agricultural Colleges Branch of China Association for Educational Technology [grant number: C25ZD01].

Data availability

The data in this work is deposited in publicly available repository, which can be found from <https://www.kaggle.com/competitions/malware-classification/data>, and the codes in this study are available at <https://github.com/HPC-NEAU/Malware-Classification>.

References

- Alshomrani, M., Albeshri, A., Alsulami, A.A., et al., 2025. An explainable hybrid CNN-transformer architecture for visual malware classification. *Sensors* 25 (15), 4581.
- Aryal, K., Gupta, M., Abdelsalam, M., et al., 2024. A survey on adversarial attacks for malware analysis. *IEEE Access*.
- Ashawa, M., Owol, N., Hosseinzadeh, S., et al., 2024. Enhanced image-based malware classification using transformer-based convolutional neural networks (CNNs). *Electronics* 13 (20), 4081.
- Bakour, K., Ünver, H.M., 2021. VisDroid: Android malware classification based on local and global image features, bag of visual words and machine learning techniques.

- Neural Comput & Applic 33 (8), 3133–3153. <https://doi.org/10.1007/s00521-020-05195-w>.
- Chen, Q., Bridges, R.A., 2017. Automated behavioral analysis of malware A case study of WannaCry ransomware. *arXiv*, Sep. 25 [Online]. Available: <http://arxiv.org/abs/1709.08753>. (Accessed 7 March 2024).
- Dai, Y., Li, H., Qian, Y., Lu, X., 2018. A malware classification method based on memory dump grayscale image. *Digit. Invest.* 27, 30–37. <https://doi.org/10.1016/j.diin.2018.09.006>.
- Gibert, D., Mateu, C., Planes, J., 2020. The rise of machine learning for detection and classification of malware: research developments, trends and challenges. *J. Netw. Comput. Appl.* 153, 102526.
- Hassen, M., Carvalho, M.M., Chan, P.K., 2017. Malware classification using static analysis based features. In: 2017 IEEE Symposium Series on Computational Intelligence (SSCI), pp. 1–7. <https://doi.org/10.1109/SSCI.2017.8285426>.
- Inoue, D., Yoshioka, K., Eto, M., Hoshizawa, Y., Nakao, K., 2009. Automated malware analysis system and its sandbox for revealing malware's internal and external activities. *IEICE Trans. Info Syst.* E92-D (5), 945–954. <https://doi.org/10.1587/transinf.E92.D.945>.
- Janocha, K., Czarnecki, W.M., 2017. On loss functions for deep neural networks in classification. *arXiv preprint arXiv:1702.05659*.
- Jeon, S., Moon, J., 2020. Malware-detection method with a convolutional recurrent neural network using opcode sequences. *Inf. Sci.* 535, 1–15. <https://doi.org/10.1016/j.ins.2020.05.026>.
- Jeong, I.W., Lee, H.J., Kim, G.N., et al., 2025. MalFormer: a novel vision transformer model for robust malware analysis. *IEEE Access*.
- Kalash, M., Rochan, M., Mohammed, N., Bruce, N.D.B., Wang, Y., Iqbal, F., 2018. Malware classification with deep convolutional neural networks. In: 2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS). IEEE, Paris, pp. 1–5. <https://doi.org/10.1109/NTMS.2018.8328749>.
- Kim, J.H., Jun, J., Zhang, B.T., 2018. Bilinear attention networks. *Adv. Neural Inf. Process. Syst.* 31.
- Kumar, S., Panda, K., 2023. SDIF-CNN: stacking deep image features using fine-tuned convolution neural network models for real-world malware detection and classification. *Appl. Soft Comput.* 146, 110676. <https://doi.org/10.1016/j.asoc.2023.110676>.
- Moskovitch, R., et al., 2008. Unknown malware detection using OPCODE representation. In: Ortiz-Arroyo, D., Larsen, H.L., Zeng, D.D., Hicks, D., Wagner, G. (Eds.), *Intelligence and Security Informatics*, Vol. 5376, Lecture Notes in Computer Science, vol 5376. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 204–215. https://doi.org/10.1007/978-3-540-89900-6_21.
- Naeem, H., et al., 2020. Malware detection in industrial internet of things based on hybrid image visualization and deep learning model. *Ad Hoc Netw.* 105, 102154. <https://doi.org/10.1016/j.adhoc.2020.102154>.
- Nataraj, L., Karthikeyan, S., Jacob, G., Manjunath, B.S., 2011. Malware images: visualization and automatic classification. In: *Proceedings of the 8th International Symposium on Visualization for Cyber Security*. ACM, Pittsburgh Pennsylvania USA, pp. 1–7. <https://doi.org/10.1145/2016904.2016908>.
- Nesmachnow, S., Toutouh, J., 2025. Adversarial attacks to image classification systems using evolutionary algorithms. *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 434–442.
- Ni, S., Qian, Q., Zhang, R., 2018. Malware identification using visualization images and deep learning. *Comput. Secur.* 77, 871–885.
- O'Kane, P., Sezer, S., McLaughlin, K., 2011. Obfuscation: the hidden malware. *IEEE Security & Privacy* 9 (5), 41–47. <https://doi.org/10.1109/MSP.2011.98>.
- Palma Salas, M.I., De Geus, P., Botacin, M., 2023. Enhancing malware family classification in the microsoft challenge dataset via transfer learning. In: 12th Latin-American Symposium on Dependable and Secure Computing. ACM, La Paz Bolivia, pp. 156–163. <https://doi.org/10.1145/3615366.3615374>.
- Saleh, M.A., 2023. Malware detection approaches based on operational codes (OpCodes) of executable programs: a review. *Int. J. E. Enterpren. Innovat.* 11 (2), 570–585. <https://doi.org/10.52549/ijeiv.1112.4454>.
- Singh, A., Dutta, D., Saha, A., 2019. MIGAN: malware image synthesis using GANs. *AAAI* 33 (1), 10033–10034. <https://doi.org/10.1609/aaai.v33i01.330110033>.
- Vasan, D., Alazab, M., Wassan, S., Naeem, H., Safaei, B., Zheng, Q., 2020. IMCFN: image-Based malware classification using fine-tuned convolutional neural network architecture. *Comput. Netw.* 171, 107138. <https://doi.org/10.1016/j.comnet.2020.107138>.
- Xiang, E.W., Cao, B., Hu, D.H., Yang, Q., 2010. Bridging domains using world wide knowledge for transfer learning. *IEEE Trans. Knowl. Data Eng.* 22 (6), 770–783.
- Yakura, H., Shinokaki, S., Nishimura, R., Oyama, Y., Sakuma, J., 2018. Malware analysis of imaged binary samples by convolutional neural network with attention mechanism. In: *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, pp. 127–134.
- Zhang, H., Xiao, X., Mercaldo, F., Ni, S., Martinelli, F., Sangaiah, A.K., 2019. Classification of ransomware families with machine learning based on N-gram of opcodes. *Future Gener. Comput. Syst.* 90, 211–221. <https://doi.org/10.1016/j.future.2018.07.052>.
- Zhou, C., Li, Z., Song, J., Xiang, W., 2024. Transvae-dta: transformer and variational autoencoder network for drug-target binding affinity prediction. *Comput. Methods Progr. Biomed.* 244, 108003.
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., et al., 2020. A comprehensive survey on transfer learning. *Proc. IEEE* 109 (1), 43–76.
- Zou, B., Cao, C., Wang, L., Fu, S., Qiao, T., Sun, J., 2024. FACILE: a capsule network with fewer capsules and richer hierarchical information for malware image classification. *Comput. Secur.* 137, 103606. <https://doi.org/10.1016/j.cose.2023.103606>.
- Zou, B., Cao, C., Tao, F., et al., 2025. JPEG-domain malware detection with pretrained lightweight vision transformer model. *IEEE Trans. Dependable Secure Comput.*
- BIG Malware Dataset from Microsoft. Accessed: April. 1, 2024. [Online]. Available: <https://www.kaggle.com/datasets/muhammad4hmed/malwaremicrosoftbig>.
- Microsoft malware classification challenge (BIG 2015). Accessed: April. 1, 2024. [Online]. Available: <https://kaggle.com/competitions/malware-classification>.